

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Отчет по лабораторной работе №2.

Выполнил: студент группы БВТ2402

Скопа Михаил Вячеславович

Москва, 2026

Цель работы: познакомить с основами работы с инструментом сборки Gradle.

Задание для выполнения лабораторной работы

1. Ознакомьтесь с теоретической частью и создайте новый проект Gradle, как описано в методических указаниях. Изучите структуру созданного проекта и содержимое файлов `build.gradle.kts` и `settings.gradle.kts`. Добавьте в файл `build.gradle.kts` плагин `application` и укажите главный класс.

```
plugins {  
    java  
    application
```

```
application {  
    mainClass.set("org.example.Main")  
}
```

2. Найдите на сайте `mvnrepository.com` библиотеку Apache Commons Lang3. Добавьте ее в раздел `dependencies` вашего файла `build.gradle.kts` с областью видимости `implementation`. Синхронизируйте проект, чтобы зависимости загрузились. Подключите две зависимости: `ch.qos.logback:logback-classic` и `org.slf4j:slf4j-api`. Это стандартный стек логирования для Java. Найдите их актуальные версии на Maven Central.

```
dependencies {  
    implementation("org.apache.commons:commons-lang3:3.12.0")  
    implementation("ch.qos.logback:logback-classic:1.4.11")  
    implementation("org.slf4j:slf4j-api:2.0.9")  
  
    implementation(project(":string-utils"))  
  
    testImplementation(platform("org.junit:junit-bom:5.9.1"))  
    testImplementation("org.junit.jupiter:junit-jupiter")  
}
```

Подключил зависимости, область `implementation` означает, что библиотека нужна для компиляции и работы данного модуля.

3. Создайте в пакете `org.example` класс `Main` (если его нет). Инициализируйте логгер, в методе `main` напишите код, который выводит приветствие в консоль, запрашивает у пользователя ввод строки. Используя методы из библиотеки

Apache Commons Lang3 (например, `StringUtils.reverse()` или `StringUtils.capitalize()`), выполните какое-либо действие над введенной строкой. Выведите результат с помощью логгера. Добавьте запись в лог о начале и завершении работы программы.

```
public class Main {
    private static final Logger logger = LoggerFactory.getLogger(Main.class);

    public static void main(String[] args) {
        logger.info("Программа запущена");

        try (InputStream input = Main.class.getClassLoader().getResourceAsStream("build-passport.properties")) {
            Properties prop = new Properties();
            if (input != null) {
                prop.load(input);
                logger.info("Информация о сборке: Пользователь {}, ОС {}",
                    prop.getProperty("build.user"), prop.getProperty("build.os"));
                logger.info("Сборка №: {}", prop.getProperty("build.number"));
                logger.info("Git Hash: {}", prop.getProperty("build.git.hash"));
            }
        } catch (Exception ex) {
            logger.error("Не удалось прочитать паспорт сборки", ex);
        }

        Scanner scanner = new Scanner(System.in);
        System.out.print("Введите строку: ");
        if (scanner.hasNextLine()) {
            String input = scanner.nextLine();
            String result = StringProcessor.process(input);
            logger.info("Результат обработки: {}", result);
        }

        logger.info("Программа завершена");
    }
}
```

Opening Java Projects: [check d](#)
Couldn't start client Java Lang

```
package org.example.utils;

import org.apache.commons.lang3.StringUtils;

public class StringProcessor {
    public static String process(String input) {
        return StringUtils.reverse(input.trim());
    }
}
```

Добавил в логгер приветствие, работу со строкой (reverse).

4. В блок plugins файла `build.gradle.kts` добавьте `id("com.github.johnrengelman.shadow") version "8.1.1"`. Это плагин для создания fat JAR.

```
mirphon@mirphon:~/Test-Gradle/app/build/libs$ java -jar app-1.0-SNAPSHOT-all.jar
00:28:53.400 [main] INFO org.example.Main -- Программа запущена
00:28:53.405 [main] INFO org.example.Main -- Информация о сборке: Пользователь mirphon, ОС Linux
00:28:53.408 [main] INFO org.example.Main -- Сборка %: 24
00:28:53.408 [main] INFO org.example.Main -- Git Hash: eacbf45
Введите строку: jfje
00:28:57.496 [main] INFO org.example.Main -- Результат обработки: ejfj
00:28:57.496 [main] INFO org.example.Main -- Программа завершена
mirphon@mirphon:~/Test-Gradle/app/build/libs$ |
```

Плагин работает.

5. Создайте Gradle-задачу с именем generateBuildInfo, которая будет создавать в ресурсах проекта файл build-passport.properties со следующей информацией:

- Имя пользователя, который запустил сборку (из переменной окружения USERNAME или USER)
- Название операционной системы
- Версия Java, на которой работает Gradle
- Текущая дата и время сборки
- Произвольное приветственное сообщение

```

abstract class GenerateBuildInfoTask : DefaultTask() {

    @get:Input
    abstract val gitCommitHash: org.gradle.api.provider.Property<String>

    @TaskAction
    fun generate() {
        val counterFile = project.file("build-counter.txt")
        if (!counterFile.exists()) counterFile.writeText("0")

        val currentCount = counterFile.readText().trim().toInt()
        val nextCount = currentCount + 1
        counterFile.writeText(nextCount.toString())

        val outputFile = project.file("src/main/resources/build-passport.properties")
        outputFile.parentFile.mkdirs()

        val username = System.getenv("USERNAME") ?: System.getenv("USER") ?: "Unknown"
        val os = System.getProperty("os.name")
        val javaVer = System.getProperty("java.version")
        val timestamp = LocalDateTime.now().toString()

        outputFile.writeText("""
            build.number=$nextCount
            build.user=$username
            build.os=$os
            build.java.version=$javaVer
            build.date=$timestamp
            build.git.hash=${gitCommitHash.get()}
        """.trimIndent())

        println("Build #${nextCount} generated with hash: ${gitCommitHash.get()}")
    }
}

val generateBuildInfo = tasks.register<GenerateBuildInfoTask>("generateBuildInfo") {
    gitCommitHash.set(providers.exec {
        CommandLine("git", "rev-parse", "--short", "HEAD")
    }.standardOutput.asText.map { it.trim() }.orElse("no-git"))
}

tasks.named("processResources") {
    dependsOn(generateBuildInfo)
}

```

```

mirphon@mirphon:~/Test-Gradle$ gradle generateBuildInfo

```

```

> Task :app:generateBuildInfo
Build #25 generated with hash: eacbf45

```

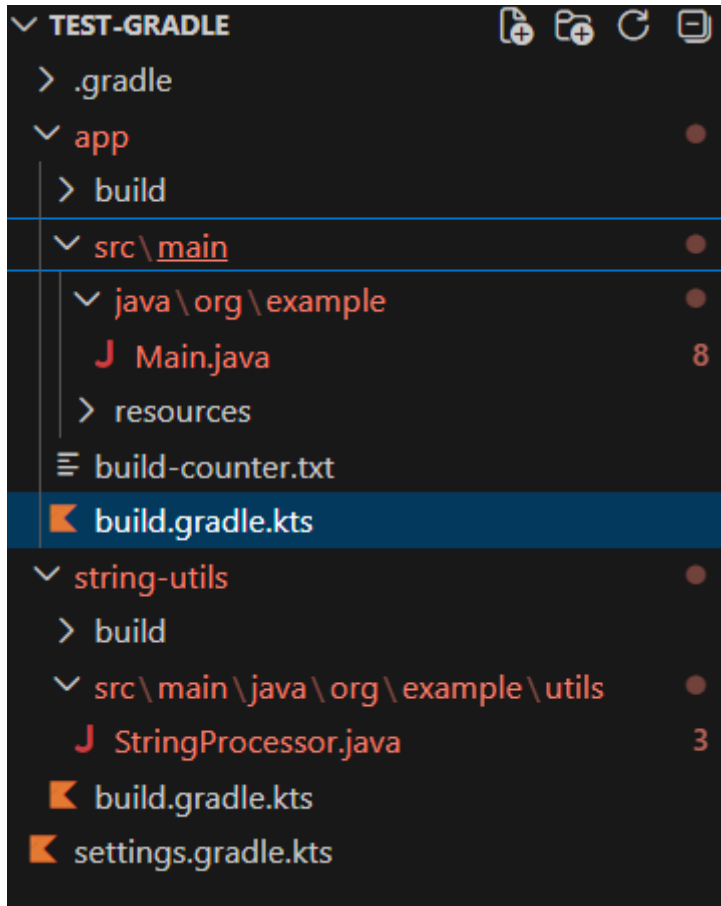
```

BUILD SUCCESSFUL in 871ms
1 actionable task: 1 executed
mirphon@mirphon:~/Test-Gradle$ |

```

6. Дополнительное задание 1

Создайте простую многомодульную структуру, выделив логику работы со строками в отдельную библиотеку. Выполните команду `gradle build`. Обратите внимание, как Gradle собирает сначала модуль `string-utils`, а потом основной проект, связывая их.



Создал простую многомодульную структуру.

```
mirphon@mirphon:~/Test-Gradle$ gradle build

> Task :app:generateBuildInfo
Build #26 generated with hash: eacbf45

BUILD SUCCESSFUL in 1s
13 actionable tasks: 10 executed, 3 up-to-date
mirphon@mirphon:~/Test-Gradle$ |
```

7. Дополнительное задание 2:

Добавьте в задачу `GenerateBuildInfoTask` новое поле `@Input gitCommitHash`: Property. Получите хеш последнего коммита и добавьте его в генерируемый

файл. Сделайте так, чтобы номер сборки автоматически увеличивался при каждой сборке (например, читая предыдущее значение из файла и инкрементируя его).

```
public class Main {
    private static final Logger logger = LoggerFactory.getLogger(Main.class);

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        logger.info("Программа запущена");

        try (InputStream input = Main.class.getClassLoader().getResourceAsStream(name: "build-passport.properties")) {
            Properties prop = new Properties();
            if (input != null) {
                prop.load(input);
                logger.info("Информация о сборке: Пользователь {}, ОС {}",
                    prop.getProperty(key: "build.user"), prop.getProperty(key: "build.os"));
                logger.info("Сборка №: {}", prop.getProperty(key: "build.number"));
                logger.info("Git Hash: {}", prop.getProperty(key: "build.git.hash"));
            }
        } catch (Exception ex) {
            logger.error("Не удалось прочитать паспорт сборки", ex);
        }

        Scanner scanner = new Scanner(System.in);
        System.out.print(s: "Введите строку: ");
        if (scanner.hasNextLine()) {
            String input = scanner.nextLine();
            String result = StringProcessor.process(input);
            logger.info("Результат обработки: {}", result);
        }

        logger.info("Программа завершена");
    }
}
```

```

abstract class GenerateBuildInfoTask : DefaultTask() {

    @get:Input
    abstract val gitCommitHash: org.gradle.api.provider.Property<String>

    @TaskAction
    fun generate() {
        val counterFile = project.file("build-counter.txt")
        if (!counterFile.exists()) counterFile.writeText("0")

        val currentCount = counterFile.readText().trim().toInt()
        val nextCount = currentCount + 1
        counterFile.writeText(nextCount.toString())

        val outputFile = project.file("src/main/resources/build-passport.properties")
        outputFile.parentFile.mkdirs()

        val username = System.getenv("USERNAME") ?: System.getenv("USER") ?: "Unknown"
        val os = System.getProperty("os.name")
        val javaVer = System.getProperty("java.version")
        val timestamp = LocalDateTime.now().toString()

        outputFile.writeText("""
            build.number=$nextCount
            build.user=$username
            build.os=$os
            build.java.version=$javaVer
            build.date=$timestamp
            build.git.hash=${gitCommitHash.get()}
        """.trimIndent())

        println("Build #${nextCount} generated with hash: ${gitCommitHash.get()}")
    }
}

val generateBuildInfo = tasks.register<GenerateBuildInfoTask>("generateBuildInfo") {
    gitCommitHash.set(providers.exec {
        commandLine("git", "rev-parse", "--short", "HEAD")
    }.standardOutput.asText.map { it.trim() }.orElse("no-git"))
}

tasks.named("processResources") {
    dependsOn(generateBuildInfo)
}

```



```
mirphon@mirphon:~/Test-Gradle$ gradle build

> Task :app:generateBuildInfo
Build #26 generated with hash: eacbf45

BUILD SUCCESSFUL in 1s
13 actionable tasks: 10 executed, 3 up-to-date
mirphon@mirphon:~/Test-Gradle$ gradle build

> Task :app:generateBuildInfo
Build #27 generated with hash: eacbf45

BUILD SUCCESSFUL in 1s
13 actionable tasks: 10 executed, 3 up-to-date
mirphon@mirphon:~/Test-Gradle$ |
```

Добавил хэш-код и отображение номера сборки. Номер сборки меняется с каждой сборкой проекта.

Вывод: познакомился с работой с Gradle.